WORKSHOP NOTES

Prompt to Product: Build, Secure and Deploy an AI App

Plan, Build, Sync, Deploy. A step-by-step workflow using Google AI Studio, GitHub and Vercel, with a real-world inventory app as the project.

Contents

1. The method at a glance
2. Phase 1: Build the app in AI Studio
3. Phase 2: Integrate API keys
4. Phase 3: Secure the app
5. Phase 4: Upload to GitHub
6. Phase 5: Deploy on Vercel
7. Updating the app later
8. Real-world app: StockWise inventory manager
9. Pre-launch checklist
10. Troubleshooting
11. Sources and links

1. The method at a glance

Step	What you do	Where
Plan	Pick a real problem and write it as a clear prompt	AI Studio Playground
Build	Generate the app, test it, improve it one change at a time	AI Studio Build mode
Connect and secure	Add API keys as secrets, sign-in, database rules, limits	Secrets panel, Firebase
Sync	Push the code to a GitHub repository	AI Studio, GitHub tab
Deploy	Import the repo into Vercel, add environment variables, publish	vercel.com

> **Remember:** AI writes the first draft quickly. You supply the real thinking: the problem, the rules, the security, and checking that the result is correct.

2. Phase 1: Build the app in AI Studio

1. Go to **aistudio.google.com**, then **Build, then New app**.
2. Paste your prompt (the StockWise starter prompt is in Section 8) and send.
3. Wait for generation, then test in the live preview on the right.
4. Open the **Code** tab and note the frontend files and the server file.
5. Improve the app with one change per message, using the chat panel or annotation mode.
6. If something breaks, send: `Fix any build issues with the current code.`

The 4-part prompt formula

Part	Question it answers
Purpose	What is the app, and who is it for?
Inputs / outputs	What goes in, and what comes out?
Rules	How should it behave? What are the edge cases?
Look and feel	How should it look?

3. Phase 2: Integrate API keys

Gemini

Nothing to add. AI Studio stores `GEMINI_API_KEY` as a server-side secret. You can view it under **Settings, then Secrets**.

Any other API (email, WhatsApp or SMS, maps, payments)

1. Create the key in the provider's dashboard. Restrict its permissions if the provider allows it.
2. In AI Studio, open **Settings, then Secrets** and add a secret with an uppercase name, for example `EMAIL_API_KEY`.
3. Give the agent this prompt:

```
Integrate the [PROVIDER] API. Read the key from the secret EMAIL_API_KEY on the server side only. Create a server endpoint /api/send-order that validates the input (supplier email format, message max 1000 characters), calls the provider, and returns only success or a friendly error. The frontend must call my endpoint and never the provider directly. Never log or expose the key.
```

4. Test a valid input, an empty input, and a failure case (temporarily use a wrong key).

Rule: A key belongs in a server-side secret or environment variable. Never type it into frontend code, a chat, a screenshot, or GitHub.

4. Phase 3: Secure the app

1. **Sign-in:** ask for *Add Sign in with Google. Each user must only see their own inventory.*
2. **Database rules:** if you use Firestore, ask the agent to set security rules so users can read and write only their own documents. Test with two different Google accounts. If you can see the other account's data, the rules are wrong.
3. **Usage limits:** ask for *Limit each user to 30 AI requests per day and show a friendly message when reached.*
4. **Input validation:** ask for *Validate all inputs on the server: numbers must be non-negative, text max 100 characters, CSV uploads max 2 MB and .csv only.*
5. **Run the review prompt** below.
6. **Check for leaks yourself:** open browser developer tools, go to the Network tab, use the app, and confirm no key appears in any request or in the page source.

SECURITY REVIEW PROMPT

```
Review this app for security and fix the issues. Check that: no API keys appear in client-side code; all external API calls happen on the server; user input is validated; each user can only access their own data; AI calls have a per-user daily limit; error messages do not reveal internal details. List what you changed and anything I must configure manually.
```

Extra protections

- **Prompt injection:** treat uploaded text as data, not instructions. Never put secrets in a prompt. Require a human click before risky actions such as sending messages or deleting data.
- **Costs:** set quota and billing alerts for Gemini and any paid API.
- **Web basics:** HTTPS (Vercel provides it), restricted CORS, friendly error messages, update dependencies.
- **Privacy:** collect only the data you need, and add a short note on what is stored.

> **Be honest:** the agent's review is a starting point, not proof the app is secure. Read the Code tab and test with a second account yourself.

5. Phase 4: Upload to GitHub

1. Create a free account at **github.com** if you do not have one.
2. In AI Studio, open **Settings, then the GitHub tab** and connect your account.
3. Create a new repository from your app (or link an existing one).
4. Push your changes. AI Studio writes the commit messages for you.
5. Open github.com and check that the files are there.
6. Search the repo for your key text. It must not appear anywhere. Confirm no .env file was committed.

- To bring in changes made elsewhere (a local editor or a teammate), **pull** them back into AI Studio. If both sides changed a file, a "Resolve conflicts" dialog lets you choose the AI Studio or GitHub version per file.
- To start from an existing repo: in Build mode, click the **+** (Add files) menu and choose **Import from GitHub**.
- You can also download the project as a ZIP and work in your own editor.

> **Tip:** keep the repository **private** while you are still testing.

6. Phase 5: Deploy on Vercel

These steps come from general knowledge of Vercel, not AI Studio's documentation, so match them to what you see on screen.

1. Go to **vercel.com** and choose **Continue with GitHub**.
2. Click **Add New, then Project**, and **Import** your repository. If it is not listed, grant Vercel access to that repo.
3. Check the detected framework and build settings. Leave the defaults unless the build fails.
4. Open **Environment Variables** and add `GEMINI_API_KEY` (and any other secret, such as `EMAIL_API_KEY`). Create a Gemini key under **Get API key** in AI Studio.
5. Click **Deploy**. You get a public .vercel.app link.
6. If you use Google sign-in, add your Vercel domain to the authorized domains in the Firebase console, otherwise login fails on the live site.
7. Test the live link with real data, not just the preview.

If AI features fail on Vercel

AI Studio web apps use a Node server for Gemini calls, and Vercel runs backend code as serverless functions. Ask the agent:

```
Convert this project's server-side calls into Vercel serverless functions in an /api folder,
reading secrets from process.env, and update the frontend to call them.
```

Then push to GitHub again and Vercel redeploys.

Never: give a secret a VITE_ or similar public prefix. That exposes it in the browser.

7. Updating the app later

Change the app in AI Studio, push to GitHub, and Vercel redeploys on its own. Test the live link after each update.

8. Real-world app: StockWise inventory manager

Pick one niche so it feels like a real product. A small grocery store or pharmacy is a good start, because they care about both low stock and expiry dates.

What it does

- Product list with quantity, cost price, selling price, reorder level, and expiry date
- Record sales and restocks
- Dashboard: stock value, low-stock items, items expiring in 30 days, top sellers
- Reorder suggestions computed from sales history
- Supplier order drafts with a message ready to send
- CSV import and export, and a delivery-note photo that fills in stock

Why it is a strong choice

It solves a daily problem, the demo is visual, it uses a database, sign-in and an API, and a simple version can be offered to a real shop owner.

Build it in four stages

STAGE 1: CORE APP

```
Build a web app called "StockWise" for small shop owners. Users sign in with Google. They can add products (name, category, quantity, cost price, selling price, reorder level, expiry date) and record sales and restocks. Store everything in Firestore under the signed-in user. Dashboard: total stock value, low-stock items in red, items expiring within 30 days. Add CSV import and export. Clean white and green theme, large buttons, easy for non-technical users.
```

STAGE 2: SALES INSIGHT

```
Add a Sales page and a dashboard showing top 5 sellers, slow movers (no sales in 30 days), and daily sales for the last 14 days as a chart.
```

STAGE 3: FORECAST

```
For each product, calculate average daily sales over the last 30 days in code, then days of stock left = quantity / average daily sales. Suggest a reorder quantity = average daily sales x (supplier lead time days + 7 safety days) minus current stock. Let me set the lead time per product. Show a "Reorder soon" list sorted by days left. Use Gemini only to write a short plain-language explanation of the list. Do all numbers in code, not with the AI.
```

STAGE 4: SUPPLIER ORDERS AND PHOTO INTAKE

```
Add a "Create supplier order" button that groups low-stock items by supplier and drafts an
order message. Add a delivery-note upload: Gemini reads item names and quantities from the
photo, shows them in an editable table, and only updates stock after the user confirms.
```

Design rule: do the calculations (days of stock left, reorder quantity) in code and use Gemini only to explain them. Require user confirmation before a photo changes stock, because AI can misread a blurry note.

Other inventory-style ideas

- **Expiry tracker** for pharmacies and cafes, listing items by expiry date
- **Customer ledger (khata) tracker** showing who owes how much, with reminder drafts
- **Supplier price comparison** highlighting the cheapest quote per item
- **Asset and maintenance tracker** for workshops and clinics

9. Pre-launch checklist

- [ ] Two-account test passes (users cannot see each other's data)
- [ ] No key in the Network tab, page source, or GitHub repo
- [ ] Usage limit and billing alerts set
- [ ] Bad input and failed API calls handled with friendly messages
- [ ] Google sign-in works on the live Vercel domain
- [ ] Live Vercel link tested end to end with real, messy data
- [ ] A short privacy note added

10. Troubleshooting

Problem	What to do
Preview shows a blank screen or error	Send "Fix any build issues with the current code." If it persists, paste the error into the chat.
Vercel build fails	Open the build log, copy the error to the agent, and ask it to fix the project for Vercel.
App loads on Vercel but AI features fail	Check that GEMINI_API_KEY is set in Vercel, then convert server calls to serverless functions (Section 6).
Google sign-in fails on the live site	Add your Vercel domain to the authorized domains in the Firebase console.
Key appears in the Network tab	Move the call to the server, remove the public prefix, revoke the key and create a new one.
Quota used up or unexpected bill	Add sign-in and per-user limits, and set quota and billing alerts.

11. Sources and links

- **Build apps in Google AI Studio (GitHub sync, secrets, deployment)**
  https://ai.google.dev/gemini-api/docs/aistudio-build-mode
- **Google AI Studio**
  https://aistudio.google.com
- **GitHub**
  https://github.com
- **Vercel**
  https://vercel.com

> **Note:** AI Studio and Vercel update often. Menus, limits and features may change, so check the official docs for the latest details.